FL-02: Prompting Fundamentals on Real Tasks v2

Task: Tailoring a technical cover letter for a target ML role **Phase:** Foundations | Est. hours: 6

---

1. The Task

Write a cover letter for an ML Engineer / evaluation-focused role, using my real background (FlyRank AI internship, StudyGapAI hackathon win, SolarSense project, Alzheimer's CNN data-integrity catch, 7+ years tutoring) mapped against a real job description.

I used a real target JD for the later iterations: **Ground-Truth Expert / ML Practitioner for Agentic Evaluation Benchmarks**, a role authoring multi-step ML tasks (modifying RL reward computation, running training experiments, analyzing model failures) and evaluating frontier models in tight feedback loops with researchers.

---

2. Six Versions

Naive baseline — no technique

Prompt:

"Please write me a cover letter for an ML engineer role."

Output (excerpt):

"I am writing to express my interest in the Machine Learning Engineer position at [Company Name]. With a strong foundation in machine learning, software engineering, and data-driven problem solving, I am excited about the opportunity to build and deploy intelligent systems that solve meaningful real-world problems… I bring a combination of analytical thinking, engineering discipline, and a willingness to learn quickly."

Note: Standard boilerplate. Generic [Company Name] placeholder, no real accomplishments, no target JD. Could describe literally any ML candidate applying anywhere. This is the "before" I was starting from.

Version 1 — Role assignment

Technique: System persona ("Act as an expert Principal Technical Recruiter and AI Career Strategist")

Prompt:

"Act as an expert Principal Technical Recruiter and AI Career Strategist. Write a cover letter for an ML Engineer position applying to a frontier AI lab evaluation team."

Output (excerpt):

"I see evaluation as a core part of building trustworthy AI—not simply a measurement layer applied after training... Effective evaluations need to withstand optimization pressure: if a benchmark becomes a target, models will eventually learn to exploit its weaknesses."

What changed and why: Adding the persona alone — no CV, no JD — shifted the entire register of the letter. Without any of my real accomplishments in the prompt, the model still stopped writing generic "I am excited to apply" filler and started writing about evaluation as a discipline: benchmark saturation, adversarial test sets, optimization pressure on metrics. The persona changed *what the model considered worth talking about*, before I'd given it anything specific about me. The cost: since I hadn't supplied real background yet, the strong-sounding technical claims ("comfortable working across the model-development lifecycle") were generic assertions with nothing under them — confident-sounding but not yet grounded in anything I'd actually done.

Version 2 — Context and data injection

Technique: Real CV accomplishments + real JD text inserted into the prompt

Prompt: Same persona, now with my actual background (FlyRank AI, StudyGapAI hackathon — 287-line system prompt enforcing strict JSON output under 3–5s latency, SolarSense Flask gateway across 20 cities, the Alzheimer's CNN 99.83% accuracy anomaly and rejection, 7+ years tutoring) and the actual Ground-Truth Expert / Cincinnatus JD pasted in.

Output (excerpt):

"While conducting CNN architecture research for Alzheimer's classification, I encountered a seemingly impressive 99.83% accuracy result. Rather than accepting it, I investigated the data pipeline and identified source pre-augmentation as the cause of the anomaly. I rejected the result and rebuilt the experiment using a clean retraining pipeline."

What changed and why: This is the single largest jump in the whole ladder. V1's generic lifecycle claims were replaced by specific, checkable facts — the 99.83% anomaly, the 287-line prompt, the Flask gateway across 20 cities. The letter went from *plausible* to *true*.

It also stopped being interchangeable with any other candidate's letter: nobody else has this exact anomaly story. The remaining problem was presentation, not content — it was still one dense wall of text a recruiter has to read start to finish to find the good parts.

Version 3 — Few-shot structural exemplar

Technique: <example> block showing the target 3-part skeleton (Why Me / Why This Role & Lab / Future Impact)

Prompt: Same context as V2, plus:

<example> **WHY ME: PROVEN HANDS-ON RIGOR** *- [Point 1...]* *- [Point 2...]* **WHY THIS ROLE & LAB** *- [...]* **FUTURE IMPACT** *- [...]* </example>

Output (excerpt):

"**WHY ME: PROVEN HANDS-ON RIGOR** ... **WHY THIS ROLE & LAB** The Ground-Truth Expert / ML Practitioner role at Cincinnatus / Frontier AI Lab strongly aligns with how I want to apply my ML skills: not simply building models, but stress-testing what models can actually do..."

What changed and why: The output reorganized itself into exactly the three labeled sections from the exemplar, in the same order, with bolded headers. A recruiter can now scan in five seconds and find "why hire this person" without reading the whole letter. This is a pure structure change — the underlying facts (99.83%, hackathon, Flask gateway) are the same ones from V2, just sorted into a skimmable shape. What it didn't fix: there was still no hard word ceiling, so some sections ran long, and a few generic connective sentences ("I would welcome the opportunity...") survived between the sharp specific claims.

Version 4 — Output structure lockdown + negative constraints

Technique: XML output tags, strict word ceiling, explicit banned-word list

Prompt: Same as V3, plus:

"Output must be enclosed within <cover_letter> tags with sub-sections <why_me> <why_lab> <future_impact>. Maximum 350 words. Do NOT use cliché buzzwords like 'passionate self-starter', 'synergy', 'hardworking', or 'dream job'. Highlight specific technical facts (287-line prompt, 99.83% accuracy rejection, 6-day hackathon ship, Flask gateway across 20 cities)."

Output (excerpt):

"I don't produce clean-looking results and stop there — I interrogate them... **99.83% accuracy** was artificially inflated by source-level pre-augmentation. I rejected the result and rebuilt the retraining pipeline around clean data rather than optimizing for a misleading metric."

What changed and why: Two visible effects. First, every remaining generic filler phrase from V3 is gone — no "welcome the opportunity," no soft enthusiasm language — because the ban list gave the model a concrete negative target instead of a vague "sound less corporate" instruction. Second, the parseable <cover_letter> / <why_me> / <why_lab> / <future_impact> tag structure means this output could be piped straight into an ATS-style template or a script, not just read by a human. The word ceiling forced real compression: dense sentences carrying two facts each instead of one fact per sentence. What was still missing: the model was executing my structure well, but it hadn't reasoned about *why* these particular facts matter to *this specific* JD before writing — it was following the template, not diagnosing the role.

Version 5 — Step decomposition (<scratchpad> reasoning)

Technique: Mandatory pre-writing reasoning stage — friction-point analysis, evidence ranking, editorial-cut justification — before any draft text is produced

Prompt: Same as V4, plus:

"Before writing the final cover letter, perform the following steps inside <scratchpad> tags: Step 1: Analyze the JD's primary pain point and map it to my specific experience. Step 2: Outline the key evidence points that prove I am a 'doer' who cares about ground-truth accuracy. Step 3: Draft the structured cover letter following all section tags and word count constraints."

Output (scratchpad excerpt):

"The core pain point in this JD isn't 'can you build ML systems' — it's 'can frontier labs trust the ground-truth data this person produces...' Frontier models increasingly pass shallow benchmarks by pattern-matching or exploiting evaluation artifacts (reward hacking, leaked test signal, spurious correlations) — so the highest-value hire isn't the best coder, it's the person structurally predisposed to distrust a clean-looking result."

Final letter (excerpt):

"I don't produce clean-looking results and stop there — I interrogate them. During a CNN architecture search for Alzheimer's classification, I flagged a 99.83% accuracy result as an anomaly, traced it to source pre-augmentation leakage in the training pipeline, and rejected the number in favor of a clean retraining run. That instinct — distrust the

flattering result until you've found why it's flattering — is the same instinct this role requires..."

What changed and why: This is a different kind of improvement from V1–V4, which all changed *how the letter looked*. V5 changed *what the letter argued*. The scratchpad explicitly reframed the JD's real ask — not "can this person code" but "can this person be trusted not to fabricate ground truth" — before a single sentence of the letter was written. That reframe then shows up in the final text: the 99.83% story is no longer just listed as a proof point, it's explicitly connected back to the JD's core risk ("reward hacking, leaked test signal") in the *Why This Role* section. The letter stopped summarizing my CV and started diagnosing the role, then answering the diagnosis. Cost: the reasoning step used part of the word/token budget and added a few "hyper-dense" phrases that read slightly more like an internal analysis note than natural interview speech — a small human-voice pass would still help before sending.

---

3. Cross-Model Comparison

The final V5 prompt (full JD + CV context + structural exemplar + output tags + word ceiling + scratchpad instruction) was run once on ChatGPT and once on Claude, so this is a same-prompt comparison, not a different-prompt one.

Context handling. ChatGPT only had what was pasted into that message — nothing more, nothing less. Claude opened its response with "I'll check what I have on file about your background and materials before drafting this," then pulled prior stored context (the "no fabrication, factual accuracy non-negotiable" CV rule) and used it to sanity-check the draft before writing. This is a real, structural difference, not a stylistic one: ChatGPT's output is only as accurate as what I remembered to paste that session; Claude's carried a persistent cross-session check I didn't have to manually re-supply.

Tone. ChatGPT's V5 output is confident and dense, marketing-adjacent even after the anti-cliché ban list — sentences like "I build, test, and ship ML systems—and I'm willing to reject results when the evidence is wrong" read as a strong opening claim. Claude's output is closer to how the fact is actually reasoned through in professional speech: "I don't produce clean-looking results and stop there — I interrogate them," followed immediately by the mechanism (traced it, rejected it, rebuilt it) rather than a summary label. The difference is asserting confidence versus demonstrating it through sequence.

Structural fidelity. Both models followed the <cover_letter> / <why_me> / <why_lab> / <future_impact> tagging exactly — no failure here on either side.

Accuracy and failure points. Neither model fabricated a number or invented a detail — both stayed inside the 99.83%, 287-line, 6-day, 20-city facts I supplied. ChatGPT's failure point was closer to genericness under pressure: even after the constraints, phrases like

"I would welcome the opportunity to discuss" survived into the final draft in earlier versions, and the "Why This Role" section leaned on restating the JD's language back rather than reframing it. Claude's failure point was verbosity in the meta-commentary: after the letter itself, it appended an unprompted list of editorial choices ("Didn't touch FlyRank… folded FlyRank's embeddings/intent classification work implicitly…") — accurate and honest about what was cut, but more explanation than a stranger applying the template would need or want by default.

Net: Claude's stored-context check produced a more provably accurate first draft with less re-supplying of facts; ChatGPT's output was competitive once the full JD+CV context was pasted manually, but required that manual re-supply every time and stayed slightly more generic in its framing language even at V5.

---

4. Final Reusable Template

<system_role>
You are an expert Principal Technical Recruiter and AI Career Strategist.
Your goal is to evaluate raw candidate context against a targeted job description and produce a ruthlessly concise, highly authoritative cover letter that focuses strictly on technical rigor, data integrity, and engineering impact.
</system_role>

<context>
<job_description>
[INSERT TARGET JOB DESCRIPTION HERE]
</job_description>

<candidate_cv>
[INSERT CANDIDATE CV / BACKGROUND DATA HERE — real accomplishments only, no placeholders]
</candidate_cv>
</context>

<structural_exemplar>
Follow this exact 3-part layout:

<example>
Why Me
[1-2 sentences: hard technical proof points, specific things shipped, and metrics from past work that prove readiness for this role's core challenges.]

Why This Role & [Company/Team]
[1-2 sentences mapping the candidate's engineering approach directly to

the specific technical mission or problem stated in the job description
— not a restatement of the JD's own language back at it.]

Future Impact
[1-2 sentences: the concrete thing the candidate will deliver from
day one, tied to the role's stated core tasks.]
</example>
</structural_exemplar>

<constraints>
- Output enclosed in <cover_letter> tags, with <why_me>, <why_role>,
  and <future_impact> sub-sections.
- Maximum 300 words total.
- No corporate boilerplate openers ("I am writing to express my
  enthusiasm...").
- Ban cliché words: "passionate," "synergy," "hardworking,"
  "results-driven," "eager," "dream job."
- Prioritize specific, checkable facts (numbers, names of things built,
  concrete outcomes) over adjectives describing the candidate.
- Never invent or round a number, a team size, or a scope detail that
  wasn't supplied in <candidate_cv>.
</constraints>

<execution_instructions>
Before writing the final letter, reason inside a <scratchpad> block:
1. Identify the job description's real underlying concern — not just
   its listed duties, but the risk or gap the hiring team is actually
   trying to close.
2. Rank the candidate's accomplishments by how directly each one
   answers that underlying concern, not by how impressive each sounds
   in isolation.
3. State explicitly which accomplishments you are cutting to fit the
   word limit, and why the cut ones are less relevant to the specific
   concern identified in step 1.
</execution_instructions>

Why this is reusable by a stranger: every field that needs a real person's facts is bracketed and named ([INSERT ... HERE]); the technique names in the constraints (role assignment, context injection, few-shot structure, output-format lockdown, step decomposition) are the same five named techniques used across V1–V5 above, so anyone applying this template is knowingly using the same ladder, not just copying a finished artifact.